Now suppose Agtron also wants the stock counts to be updated after each purchase request. What will you do? Add some more lines to the comprehension and make it harder to understand? No. As a program grows, instead of piling junk upon junk, you require more abstractions to help you manage things. Time to try classes.

A camera class describes how cameras work. In your program, camera objects are created, rendered onto a web page, and purchased. The following listing shows a `Camera` class with a `constructor` for creating cameras, a `render` method, and a `purchase` method.

Listing 5.1. A simple `Camera` class

```
class Camera
  constructor: (name, info) ->
    @name = name
    @info = info
  render: ->
    "#{@name}: #{@info.description} (#{@info.stock} in stock)"
  purchase: ->
```

The constructor function that's invoked when a new instance is created from the class.

The render method that displays the camera as a string.

The actual implementation of the purchase method is omitted in this listing.

The `new` keyword in front of a class name invokes the class `constructor`. When a `constructor` is invoked with the `new` keyword, an object is created and given to the `constructor`, where you can access it using the @ symbol. When you `new` a `Camera`,

```
x1 = new Camera 'X1', {
  description: 'An awesome camera', stock: 5
}
```

the object returned has `name` and `info` properties assigned to it by the `constructor`:

```
x1.name
#'X1'
x1.info
#{description: "An awesome camera", stock: 5}
```

The object returned also has the methods, such as `render`, from the class declaration: